RESTful Web Services with Spring Boot

HTTP, JSON, Postman, and REST Controllers

Introduction

Modern applications frequently need to exchange data with other applications, services, mobile clients, or external systems. **RESTful web services** provide a lightweight and language-independent approach to this communication, most commonly using **HTTP** as the transport protocol and **JSON** as the data format.

In this lesson, you will first examine the basic REST client-server model, then learn how HTTP requests and responses are structured, how JSON represents application data, and how Postman can be used to test REST APIs. Finally, you will build and test a simple REST endpoint using Spring Boot and `@RestController`.

Learning Objectives

By the end of this lesson, you should be able to:

- Explain the purpose of REST services.
- Describe the client-server communication model used by REST.
- Explain why REST is language-independent.
- Identify the main components of HTTP requests and responses.
- Associate common HTTP methods with CRUD operations.
- Interpret common HTTP status-code categories.
- Explain the purpose of MIME/content types.
- Describe basic JSON syntax and supported value types.
- Identify JSON objects, nested objects, and arrays.
- Use Postman to send a basic HTTP GET request.
- Inspect an HTTP response in Postman.
- Create a Spring Boot REST controller using @RestController.
- Map URLs using @RequestMapping and @GetMapping.
- Test a Spring REST endpoint using a browser and Postman.

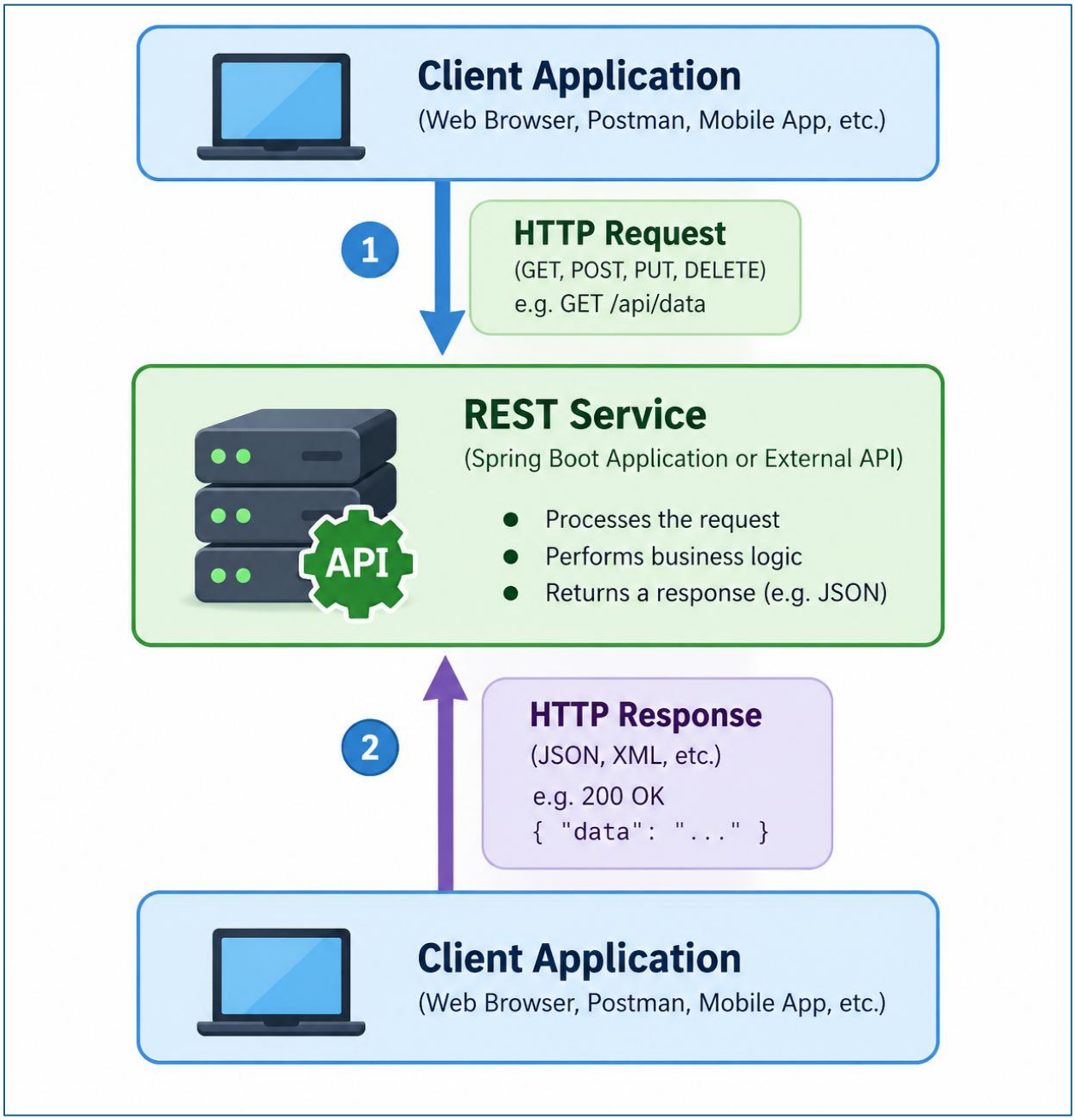
---

1. What Is REST?

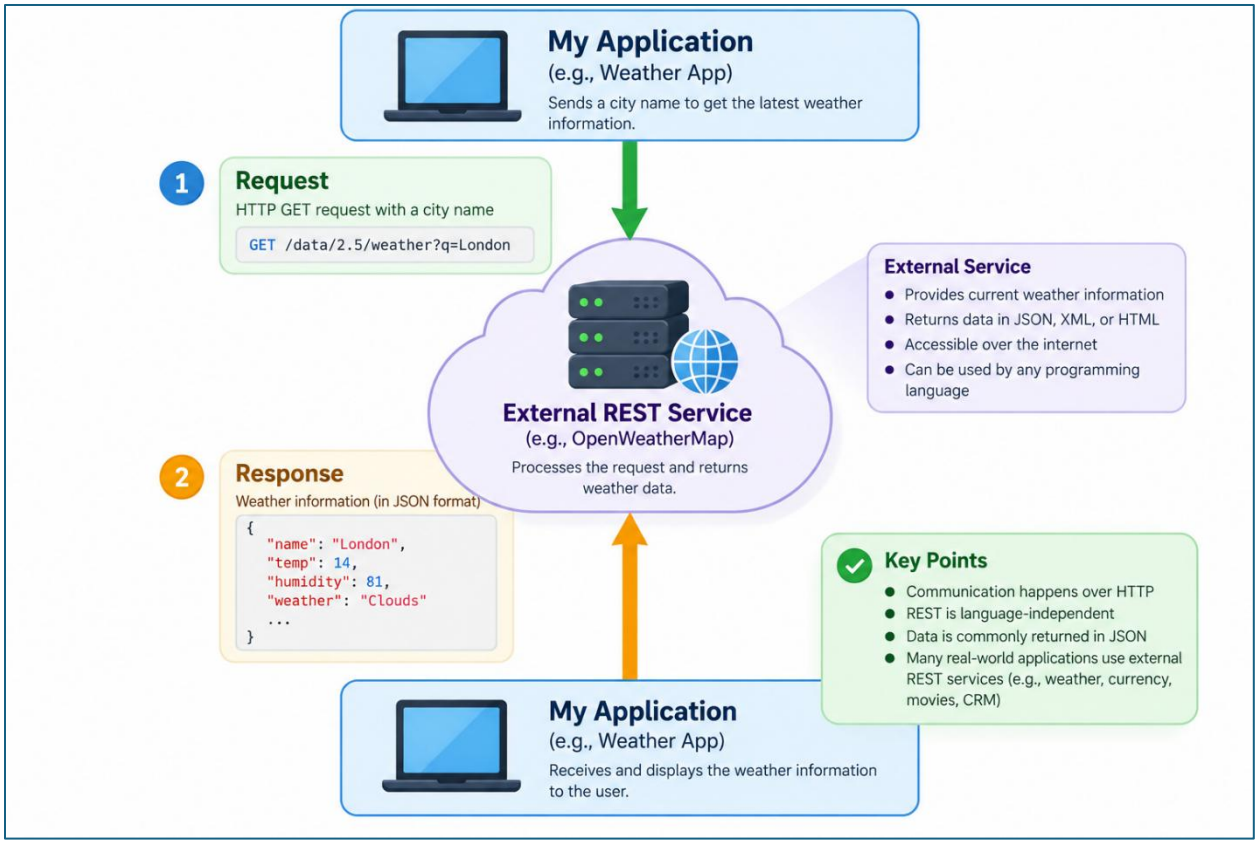
REST stands for **Representational State Transfer**.

REST is commonly used to allow separate applications to exchange information over HTTP. A client sends a request to a service, and the service processes that request and returns a response.

A simple architecture looks like this:

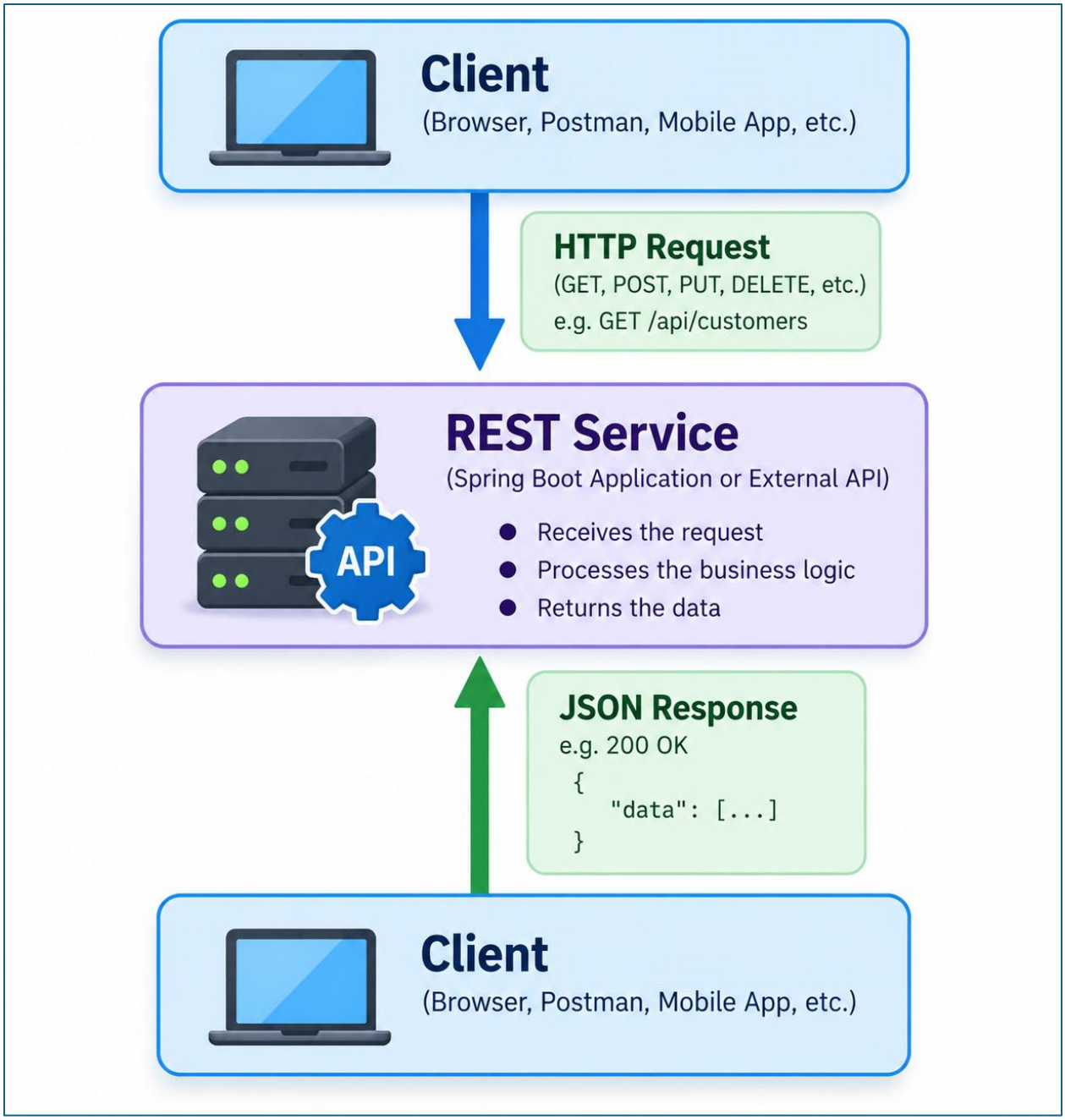


For example, a client application may request information from an external service:



The important point is that the client does not need to know how the server is internally implemented. It only needs to know:

- the endpoint to call;
- the HTTP method to use;
- what input is expected;
- what response format will be returned.

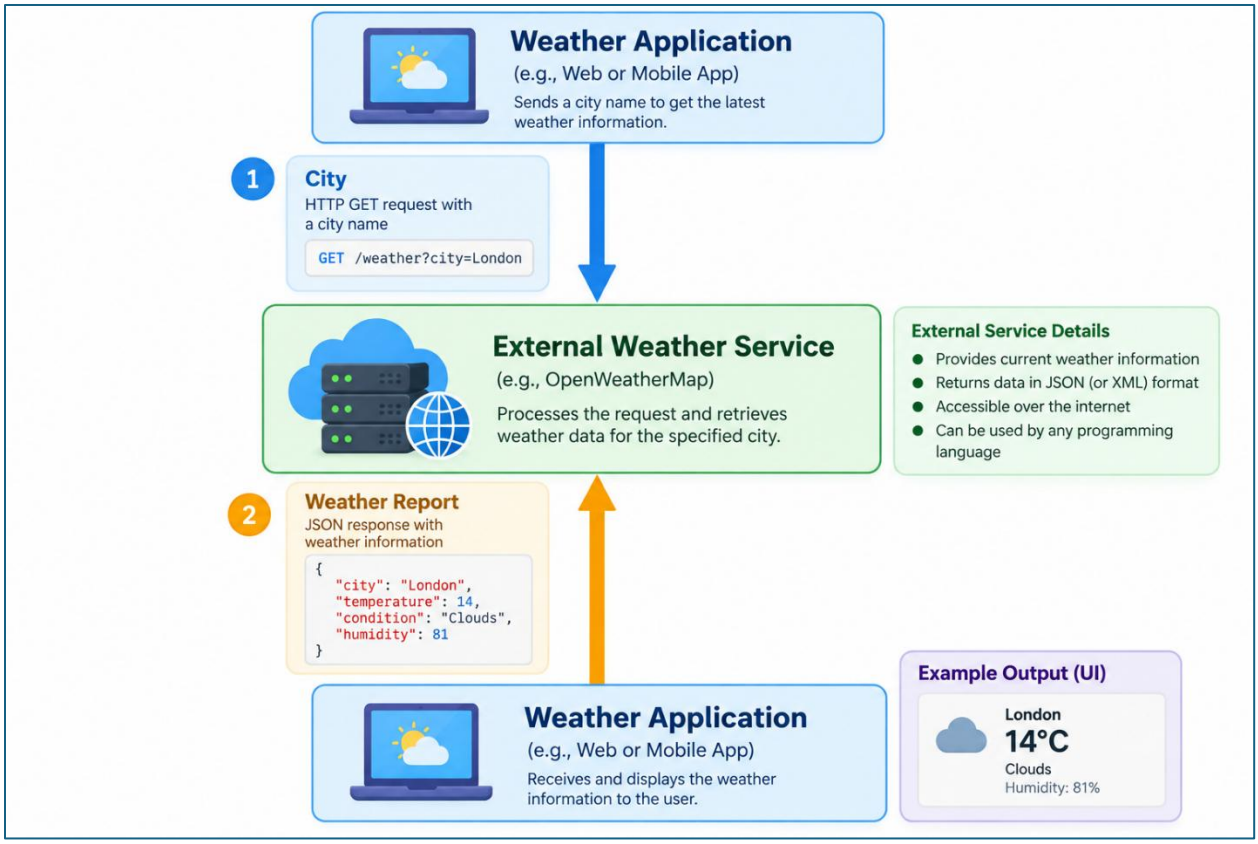


---

2. A Real-World REST Scenario

Consider an application that needs current information from an external service.

For example:



Because weather information changes continuously, the application may retrieve fresh data from an external service rather than maintaining the information locally.

This architecture raises three important questions:

1. **How do the applications communicate?**
2. **Must both applications use the same programming language?**
3. **How is the exchanged data represented?**

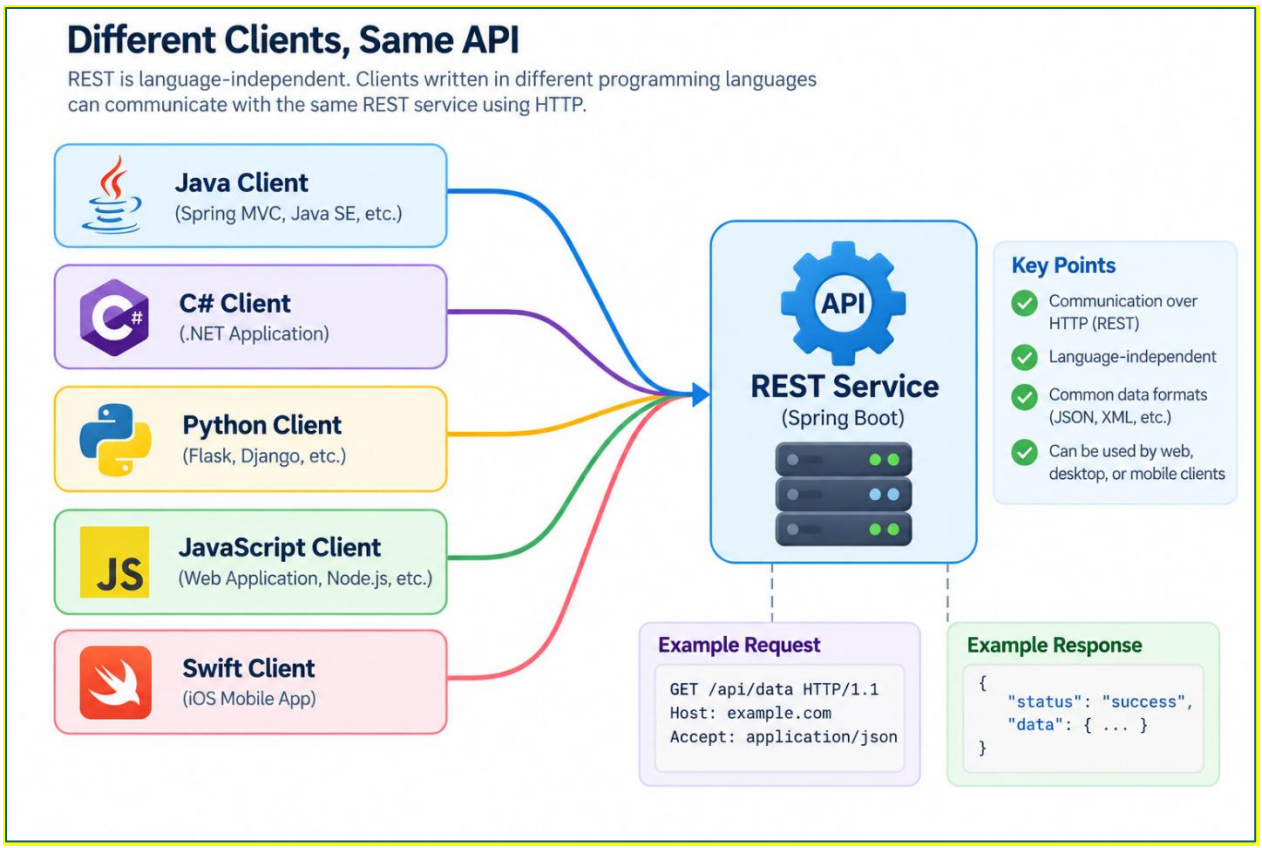
REST addresses these concerns through HTTP-based communication and standardized data formats.

---

3. REST Is Language-Independent

One of REST's major advantages is that the client and server do not need to use the same programming language.

For example:



A Spring Boot service written in Java could therefore communicate with:

- a JavaScript web application;
- a C# desktop application;
- a Python application;
- an iOS application written in Swift.

The applications communicate through the REST interface rather than sharing implementation code.

> **Note**
>
> REST provides an interoperability boundary between applications. What matters is the agreed request/response interface, not the implementation language used on either side.

---

4. REST and HTTP

REST services commonly communicate using **HTTP**.

HTTP defines several request methods that express different operations.

The four methods introduced here map naturally to the basic CRUD operations:

HTTP Method	Typical CRUD Operation	Purpose
POST	Create	Create a new entity
GET	Read	Retrieve one or more entities
PUT	Update	Update an existing entity
DELETE	Delete	Delete an existing entity

For this lesson, the practical implementation focuses mainly on:

`GET`

because we are retrieving data rather than modifying it.

> **Note**
>
> GET should be thought of as a request to **retrieve a representation of a resource**.

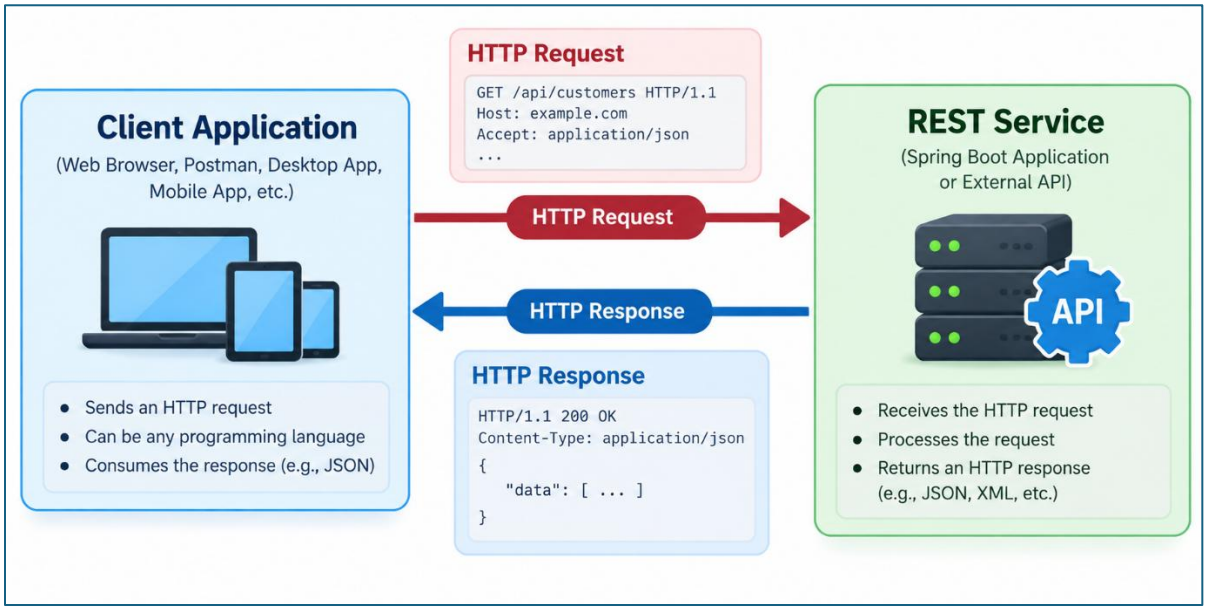
---

5. HTTP Request and Response Communication

REST communication follows a request-response model:

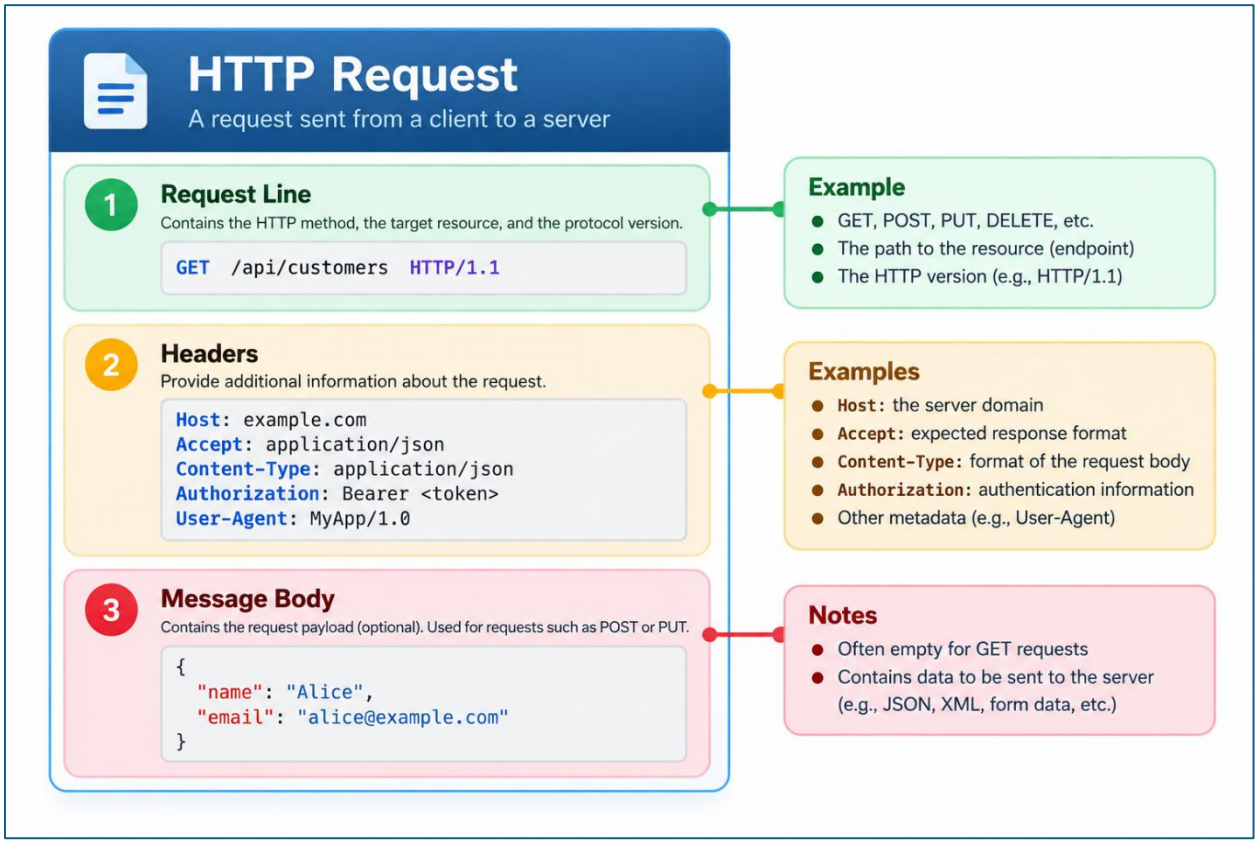
Client → HTTP Request → Server / REST Service → HTTP Response → Client

The client initiates the communication. The server receives the request, processes it, and returns a response.



6. Structure of an HTTP Request

An HTTP request can be viewed as containing three major parts:



6.1 Request Line

The request line contains information such as the HTTP method.

For example:

`GET /users`

Other possible methods include:

`POST`

`PUT`

`DELETE`

6.2 Headers

Headers contain additional information or metadata about the request.

They may provide information about:

- accepted content types;
- authentication;
- client behavior;
- other request characteristics.

6.3 Message Body

The message body contains the request payload when one is required.

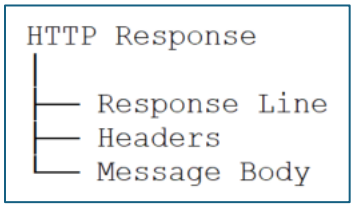
For example, requests that send data to a server may include content in the body.

> **Note**
>
> A simple GET request often does not require a request body.

---

7. Structure of an HTTP Response

An HTTP response can similarly be viewed as having three major parts:



7.1 Response Line

The response line contains the protocol information and status code.

For example: `200 OK`

7.2 Headers

Response headers provide metadata such as:

- content type;
- content size;
- other response information.

7.3 Message Body

The message body contains the actual response content.

For a REST API, this could contain JSON such as:



---

8. HTTP Status Codes

HTTP status codes tell the client what happened when a request was processed.

The major categories introduced here are:

Range	Category
100–199	Informational
200–299	Success
300–399	Redirection
400–499	Client Error
500–599	Server Error

Common examples include:

```
200 OK
404 Not Found
500 Internal Server Error
```

> **Tip**
> When testing a REST service, always inspect both the **response body** and the **HTTP status code**. A response that looks empty or unexpected often becomes easier to diagnose when the status code is checked.

---

9. MIME and Content Types

HTTP headers can describe the format of the message payload using a **content type**.

Examples include:

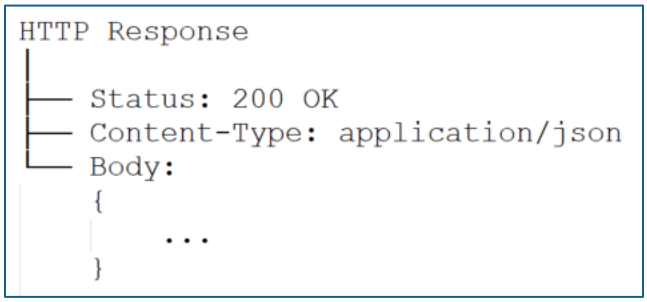
```
text/html
text/plain
application/json
application/xml
```

For REST APIs, an important content type is:

`application/json`

This indicates that the message body contains JSON data.

Conceptually:



---

10. Introduction to JSON

JSON stands for **JavaScript Object Notation**.

Despite its name, JSON is not limited to JavaScript. It is a text-based data format used for storing and exchanging information across many platforms and programming languages.

JSON can be used with:

- Java;
- C#;
- Python;
- JavaScript;
- and many other languages.

Because JSON is plain text, it can be exchanged easily between systems implemented using different technologies.

---

11. JSON Objects and Name-Value Pairs

A JSON object is enclosed in curly braces:

`{ ... }`

Inside the object, data is represented using **name-value pairs**.

For example:

```
{
  "id": 14,
  "firstName": "Mario",
  "lastName": "Rossi",
  "active": true
}
```

Each member follows the structure:

`"name" : value`

For example:

`"firstName": "Mario"`

Here:

`firstName`

is the name, while:

`Mario`

is the value.

Important

JSON names are enclosed in **double quotes**.

---

12. Common JSON Value Types

JSON values can take several forms.

Number

```
{
  "id": 14
}
```

Numbers are not enclosed in quotes.

String

```
{
  "firstName": "Mario"
}
```

Strings use double quotes.

Boolean

```
{
  "active": true
}
```

Boolean values are represented using:

```
true
```

```
false
```

Null

```
{
  "course": null
}
```

Object

A value can itself be another JSON object.

Array

A value can also contain a JSON array.

These types allow JSON to represent both simple and complex information.

---

13. Nested JSON Objects

JSON objects can contain other JSON objects.

For example:

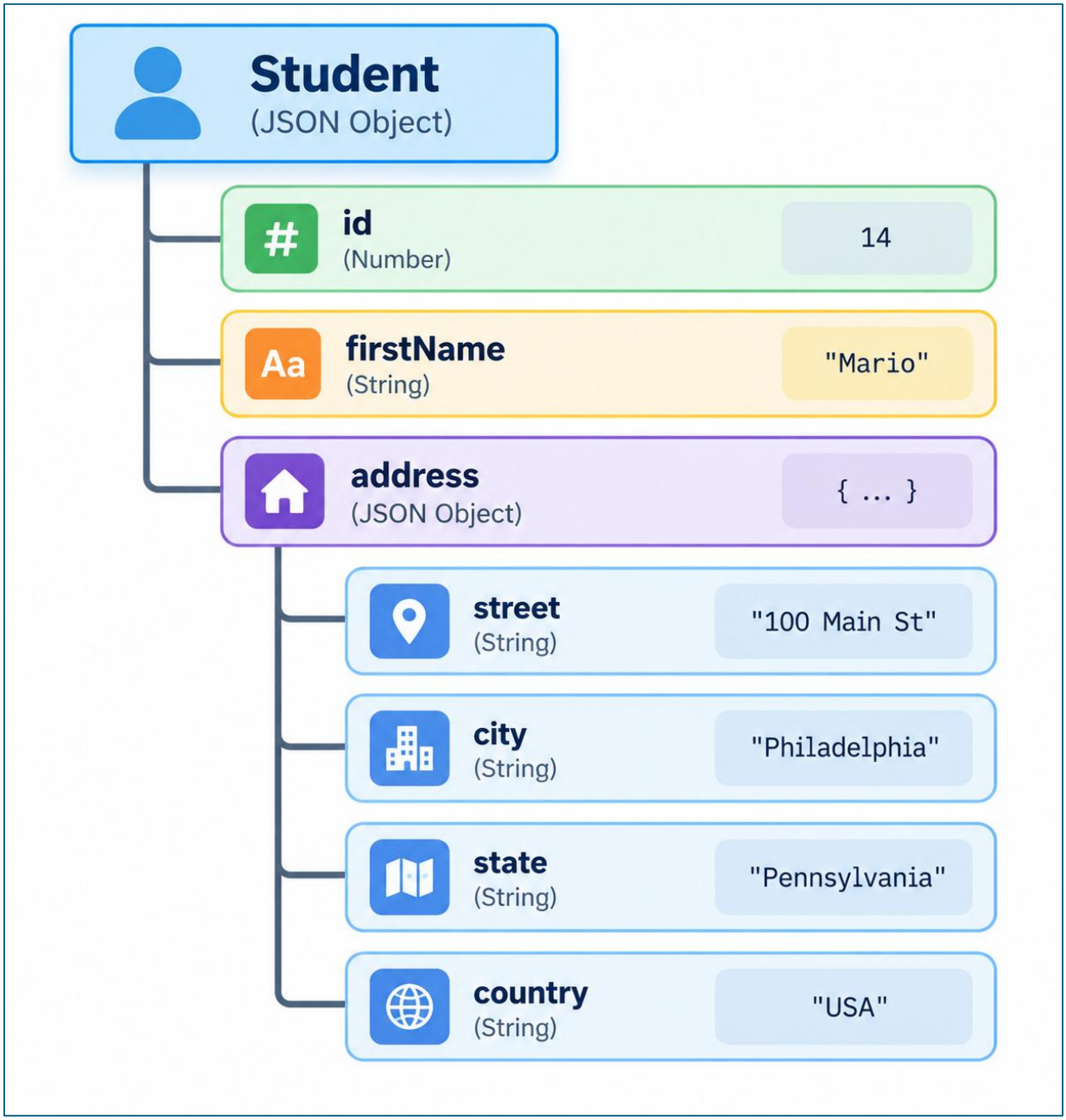
```
{
  "id": 14,
  "firstName": "Mario",
  "address": {
    "street": "100 Main St",
    "city": "Philadelphia",
    "state": "Pennsylvania",
    "zip": "19103",
    "country": "USA"
  }
}
```

The address property contains another object:

```
"address": {
    ...
```

}

This allows related information to be grouped into a structured hierarchy.



---

14. JSON Arrays

Arrays are enclosed in square brackets:

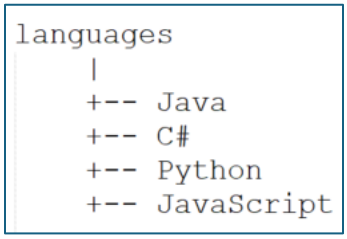
```
[ ... ]
```

For example:

```
{
  "languages": [
    "Java",
    "C#",
    "Python",
    "JavaScript"
  ]
}
```

The array contains a comma-separated sequence of values.

Conceptually:



> **Tip**
>
> Remember the distinction:
>
> `{ }` → JSON object
> `[ ]` → JSON array

---

15. REST Clients

To interact with a REST service, developers need a client capable of sending HTTP requests.

Two examples introduced here are:

- **cURL** — command-line based;
- **Postman** — graphical interface.

For simple **`GET`** requests, a browser may also be sufficient.

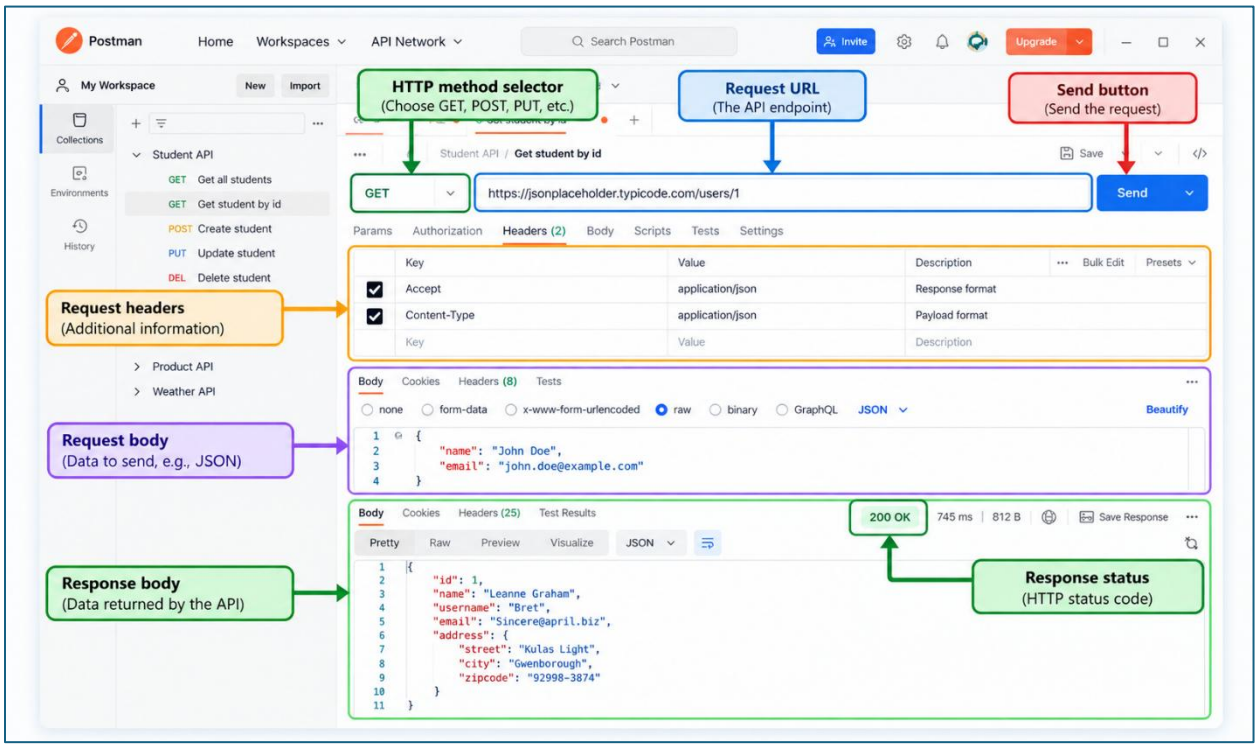
However, *Postman* provides additional information and controls that are useful when testing APIs.

---

16. Introduction to Postman

Postman provides an interface for constructing HTTP requests and examining responses.

The basic interface can be viewed as two major areas:



---

17. Practice: Send Your First GET Request with Postman

A public REST endpoint can be used to practice basic API testing.

Step 1 — Open Postman

Launch Postman.

Open a new request tab.

Step 2 — Select the HTTP Method

Choose:

`GET`

Step 3 — Enter the URL

Use the **`/users`** endpoint from **`JSONPlaceholder`**:

`https://jsonplaceholder.typicode.com/users`

Step 4 — Send the Request

Click:

`Send`

Step 5 — Inspect the Response

Observe:

- the JSON response body;
- the HTTP status;
- the response headers.

The response contains a list of users represented as a JSON array, including nested JSON structures.

---

18. Understanding the Postman Response

A successful request should show an HTTP status similar to:

`200 OK`

The response body contains JSON data.

Because the response begins with:

`[`

it represents a JSON **array**.

You can also inspect response headers and locate a content type similar to:

`Content-Type: application/json`

This indicates that the server returned JSON data.

Practical Exercise

Identify the following in Postman:

1. HTTP method.
2. Request URL.
3. HTTP status code.
4. Response body.
5. Content-Type.
6. One JSON object.
7. One nested JSON object.

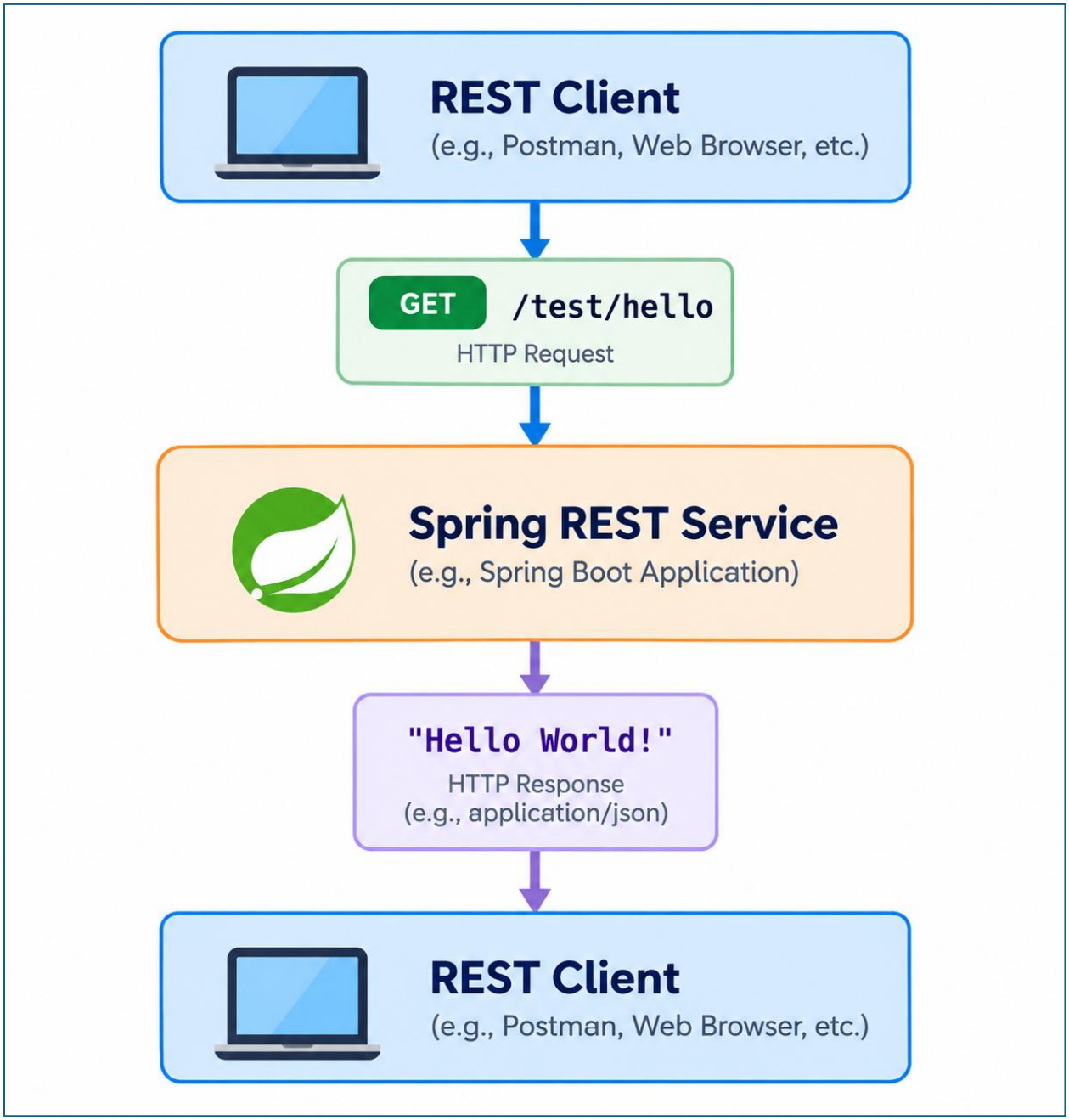
8. The outer JSON array.

---

19. From REST Concepts to Spring Boot

We now move from consuming a REST service to **creating our own REST service**.

The target interaction is:



The REST client can be:

- a browser;
- Postman.

The REST service will be implemented using Spring Boot.

---

20. Required Spring Boot Dependency

To build a REST controller, the project needs Spring Web support.

The project can include:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Alternatively, when creating the project using Spring Initializr, select the:

`Spring Web`

`dependency`.

---

21. Creating the Spring Boot Project

Step 1 — Open Spring `Initializr`

Navigate to:

`start.spring.io`

Step 2 — Configure the Project

Select appropriate settings such as:

`Project`: Maven

`Language`: Java

`Spring Boot`: appropriate version

Configure the project metadata.

Step 3 — Add Spring Web

Add:

`Spring Web`

as a dependency.

Step 4 — Generate the Project

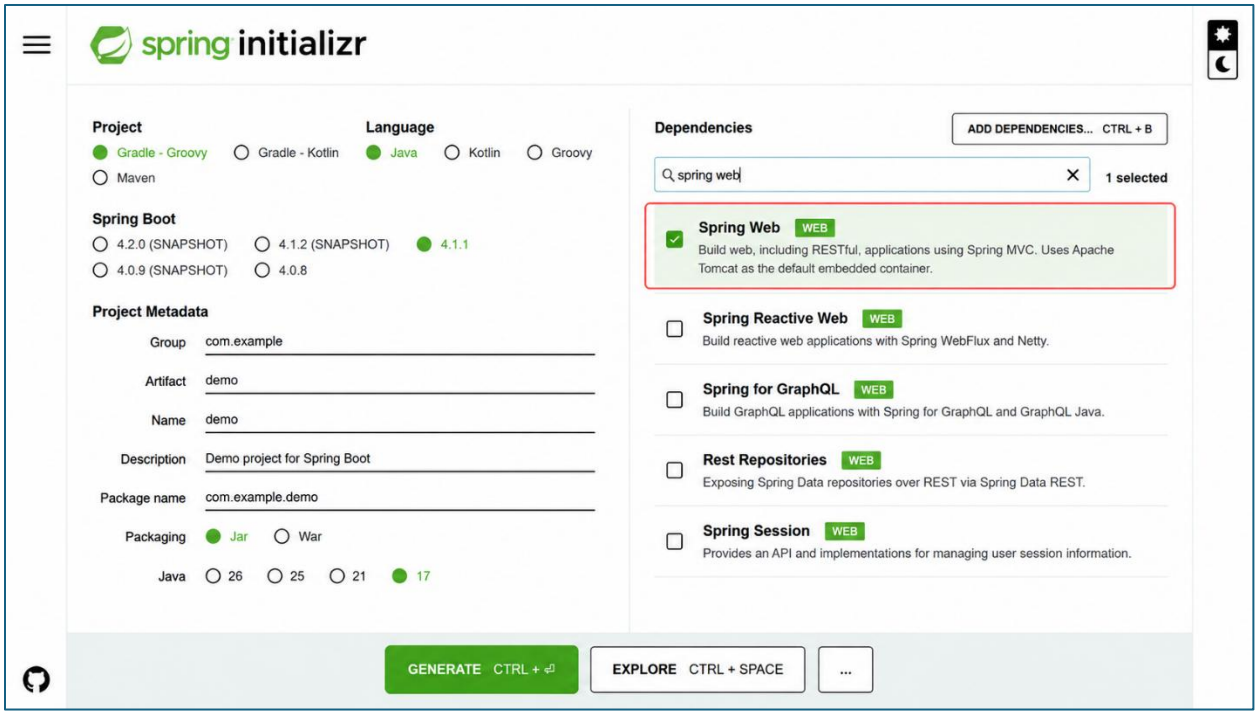
Generate and download the **`ZIP`** file.

Step 5 — Extract the Project

Unzip it into your development directory.

Step 6 — Open the Project

Open/import the project in your IDE.



---

22. Creating the REST Controller Package

Inside your application's Java source directory, create a package for REST controllers.

For example:

`com.example.demo.rest`

Inside that package, create:

`DemoRestController.java`

Keeping controllers in a dedicated package helps organize application responsibilities.

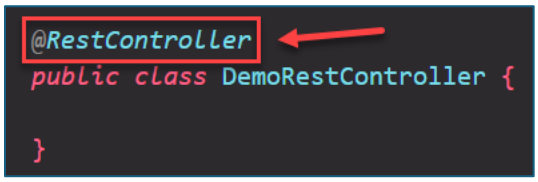
---

23. @RestController

A Spring REST controller is marked with:

`@RestController`

For example:



This annotation indicates that the class contains REST request handlers.

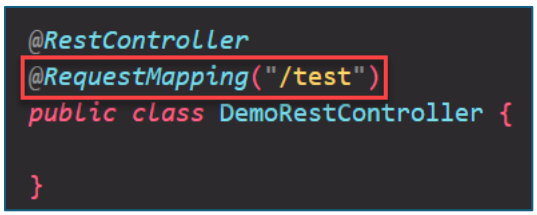
---

24. Class-Level Request Mapping

The controller can define a base path using:

`@RequestMapping("/test")`

For example:



This means the controller's endpoints will begin with:

`/test`

---

25. Creating the /hello Endpoint

Inside the controller, add:

```java
@GetMapping("/hello")
public String sayHello() {
    return "Hello World!";
}
```

The complete controller becomes:

```java
package com.example.demo.rest;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/test")
public class DemoRestController {

    @GetMapping("/hello")
    public String sayHello() {
        return "Hello World!";
    }
}
```

---

26. How the URL Is Constructed

The controller defines:

`@RequestMapping("/test")`

and the method defines:

`@GetMapping("/hello")`

Together they produce:

`/test/hello`

Conceptually:

`@RequestMapping("/test")`
`+`
`@GetMapping("/hello")`
`=`
`/test/hello`

If the application is running on localhost port 8080, the complete URL is:

`http://localhost:8080/test/hello`

Tip

When debugging a request-mapping problem, examine the mappings at both levels:

- class-level mapping;
- method-level mapping.

---

27. Understanding the Controller Code

Consider:

`@RestController`

This identifies the class as a REST controller.

Next:

`@RequestMapping("/test")`

defines the controller's base URL.

Then:

`@GetMapping("/hello")`

maps an HTTP GET request to the **`sayHello()`** method.

Finally:

```
public String sayHello() {
    return "Hello World!";
}
```

produces the response returned to the client.

The execution flow is:

```
GET /test/hello
        ↓
DemoRestController
        ↓
sayHello()
        ↓
"Hello World!"
        ↓
HTTP Response
```

---

28. Lab: Run the Spring Boot REST Service

Step 1 — Run the Application

Start the Spring Boot application from the IDE.

Confirm that the application starts successfully.

Step 2 — Open a Browser

Navigate to:

```
http://localhost:8080/test/hello
```

Step 3 — Verify the Response

You should see:

```
Hello World!
```

This confirms that:

- the embedded server received the request;
- Spring matched the URL;
- the controller method executed;
- the return value became the HTTP response.

---

29. Lab: Test the Same Endpoint with Postman

Now test the same service using Postman.

Step 1 — Open a New Request

Create a new request tab.

Step 2 — Choose GET

Select:

```
GET
```

Step 3 — Enter the Endpoint

Use:

```
http://localhost:8080/test/hello
```

Step 4 — Send the Request

Click:

`Send`

Step 5 — Inspect the Result

The response body should contain:

`Hello World!`

Also inspect the HTTP status.

A successful request should report a success status such as:

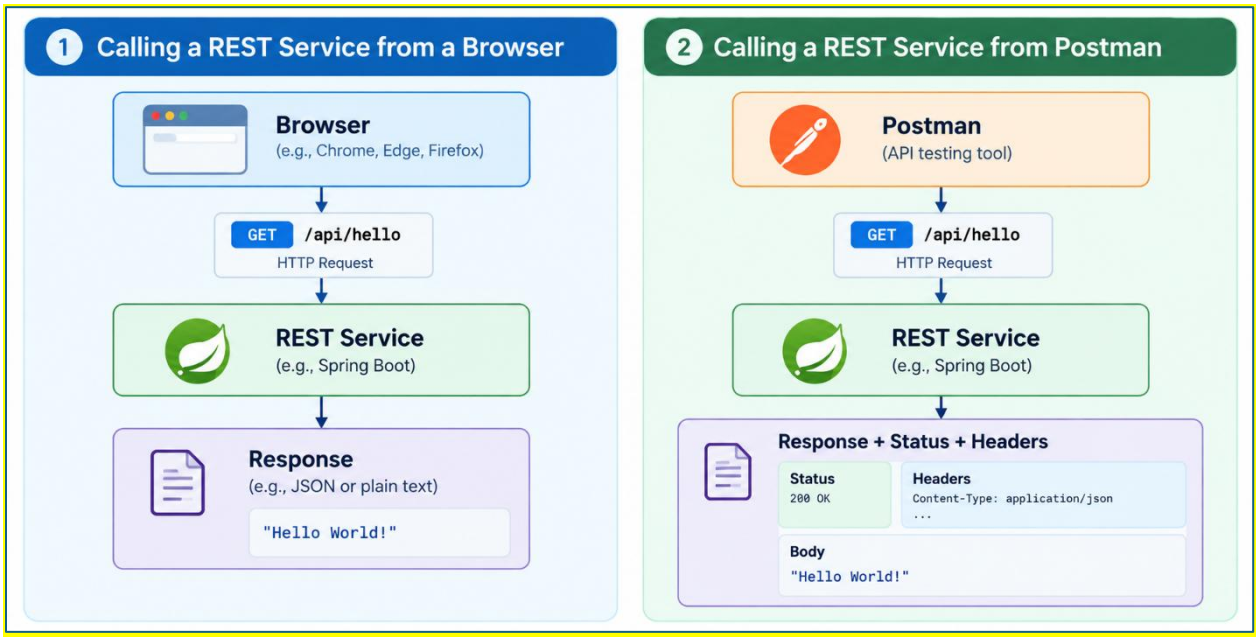
`200 OK`

The source demonstrates this same endpoint using both a browser and Postman.

---

30. Browser vs. Postman

For a simple GET endpoint, both tools can display the response:



Postman provides more information about the HTTP interaction, including:

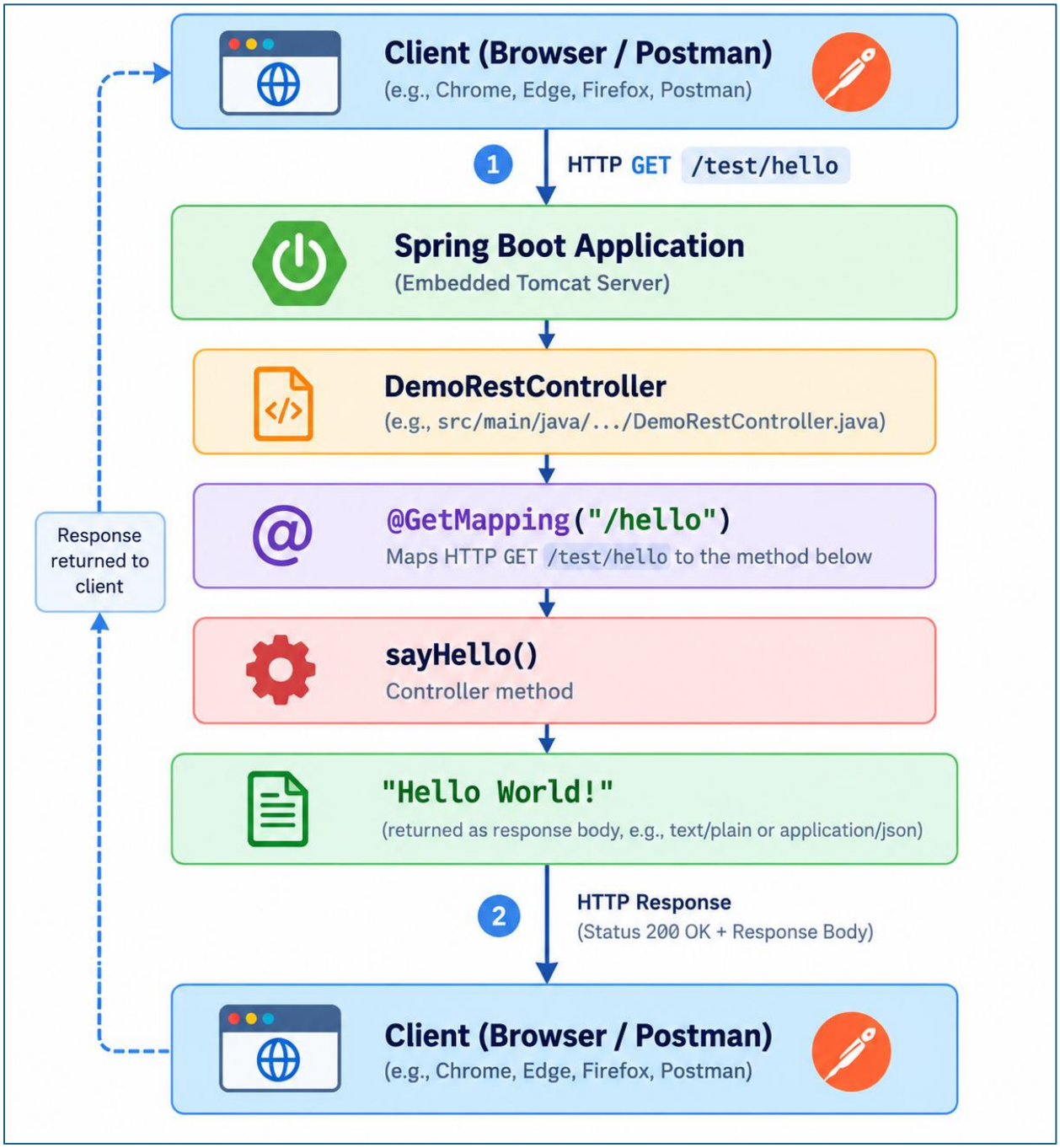
- status code;
- headers;
- response body;
- request configuration.

This makes it particularly useful as API testing becomes more complex.

---

31. End-to-End Request Flow

The complete flow for the Spring Boot example is:



---

32. Common Problems

Pitfall — Incorrect URL

If the class has:

```
@RequestMapping("/test")
```

and the method has:

```
@GetMapping("/hello")
```

the endpoint is **`/test/hello`**, not simply **`/hello`**.

Pitfall — Missing Spring Web

If the project does not contain Spring Web support, annotations and web infrastructure required for the controller may not be available.

Pitfall — Application Not Running

A correct endpoint cannot be reached if the Spring Boot application has not successfully started.

Tip

When Postman does not return the expected result, check the **HTTP method, URL, status code, and response body** before changing the Java code.

---

33. Key Concepts to Retain

REST enables applications to communicate through a standardized request-response interface, typically over HTTP. The client and server can be implemented in different programming languages because communication occurs through the API contract rather than shared implementation technology.

HTTP provides methods such as GET, POST, PUT, and DELETE, as well as structured requests, responses, headers, content types, and status codes.

JSON provides a language-independent text format composed of objects, name-value pairs, nested objects, arrays, and common values such as strings, numbers, booleans, and null.

Postman allows developers to construct HTTP requests and examine responses, including status codes, headers, and JSON bodies.

Finally, Spring Boot allows a REST service to be implemented concisely using annotations such as:

```
@RestController
@RequestMapping("/test")
@GetMapping("/hello")
```

producing an endpoint such as:

```
GET /test/hello
```

that can be tested from a browser or Postman.